Latency Side-Channel Fingerprinting of Neural Network Architectures via Inference Timing Analysis

Shlok Pandey
Manipal University Jaipur
Email: shlokpandey8219@gmail.com

***Abstract*—Latency-based side-channel leakage has emerged as a potential security risk in modern machine learning deployment environments. This paper investigates whether inference latency can be exploited to fingerprint neural network architectures in black-box settings. Different architectures, including VGG, ResNet, and Inception, exhibit distinct computational graph structures that influence inference timing behavior. The proposed methodology evaluates inference latency under controlled experimental conditions and extracts statistical timing features such as mean latency, standard deviation, and tail latency. These features are subsequently used for architecture classification using lightweight machine learning classifiers. Experimental results demonstrate that different architectures produce distinguishable timing signatures, enabling effective architecture identification using latency information alone. The findings highlight the security implications of timing leakage in machine learning systems and demonstrate that inference latency can act as a practical side-channel for neural network fingerprinting in cloud-based deployment settings.**

I. Introduction

Most cloud platforms and machine learning services provide hosted models through API-based interfaces. These services allow the user to query the system and get outputs. They often run in a black-box environment where the user has no visibility into the internal architecture. While this abstraction greatly improves usability and user access while protecting intellectual property, it also introduces potential security risks. In particular, systems may unintentionally leak information through side channels such as inference latency [3], [5], [9], power usage, memory/cache behavior, CPU/GPU utilization, packet sizes, and response sizes.

Side-channel analysis has historically shown that indirect execution signals can reveal sensitive implementation details even when the primary algorithm is not directly compromised [3]. In machine learning systems, recent work has shown that model behavior, query patterns, hardware traces, and implementation-level signals can expose information about deployed models [4]–[6], [9]. These risks are especially relevant as machine-learning-as-a-service platforms increasingly expose high-value models through black-box interfaces.

In this academic work, we focus specifically on inference latency. We methodically investigate whether the response time of neural network inference can reveal structural information about the underlying model, which can later possibly be used to create surrogate models [4], [7]. We hypothesize that differences in the computational graph structures across architectures like VGG, ResNet, and Inception can leak distinct latency signatures that can be exploited for model fingerprinting.

II. Background

A. Directed Acyclic Graphs (DAGs)

A Directed Acyclic Graph (DAG) is a directed graph that does not contain any cycles. This graph consists of directed edges, and they signify a one-way relationship or dependency between nodes. The term "acyclic" indicates that there are no cycles or any closed loops within the graph, i.e., the user cannot traverse a sequence of directed edges and return to the same node while following edge directions.

Modern neural network architectures can be modeled as DAGs, where the execution flow propagates through interconnected computational layers. Different neural network architectures exhibit different DAG structures. Sequential models such as VGG primarily show a linear computational graph, whereas architectures such as ResNet and Inception exhibit branching, merging, and parallel execution as they operate on the principles of skip connections and multi-scale feature extraction [1], [2]. These core structural differences alter the execution traits of the model, including inference latency, synchronization overhead, and computational variability. Consequently, DAG topology becomes an important factor in understanding latency-based side-channel behavior in neural network systems.

B. Neural Network Architectures

Neural network architectures vary significantly in their computational structure, execution flow, and layer organization. These differences in the architecture influence both model performance and inference characteristics.

Sequential models such as VGG are characterized by their depth [14]. VGG has deep stacks of convolutional layers arranged in a linear manner in a straightforward execution graph with minimal branching. Due to this highly sequential structure, inference propagates through a long chain of dependent operations.

In contrast, architectures like ResNet are not just one single layer like VGG; they introduce a stack of repeating residual blocks connected through skip connections, which produce a

Fig. 1. Simplified VGG-style sequential computation graph.

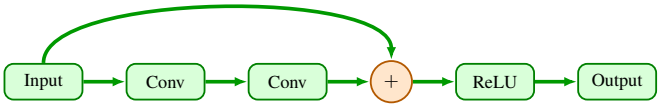

Fig. 2. Simplified ResNet residual block with skip connection and merge operation.

branch-and-merge computation graph, allowing information to bypass intermediate layers [1]. Consequently, the execution flow is no longer strictly sequential, producing a more complex Directed Acyclic Graph (DAG) structure compared to VGG.

Early architectures, like VGG, mainly got better by increasing network depth. However, increasing network depth made computation expensive. The core idea for architectures like InceptionV3 was to increase representational power without increasing computation cost by executing a module that does multi-scale feature extraction efficiently. Instead of implementing one convolution choice at a layer, i.e., a single convolutional operation, the Inception module does multiple operations in parallel on the same input, then combines them. This produces wide branching inside each module and a concatenation merge, which results in a highly parallel DAG structure [2].

C. Side-Channel Attacks

A side-channel attack is a method of extracting information from a system without compromising the primary algorithm directly. Instead of exploiting the model parameters, such as CNN weights or encryption keys, the attacker observes the execution byproducts and indirect signals produced while the system operates. Rather than targeting model parameters or architecture explicitly, side-channel attacks exploit observable characteristics like power consumption, memory access patterns, cache behavior, packet sizes, or inference latency [3], [9].

Inference latency represents one of the most examinable and practical side channels in modern machine learning systems. Different neural network architectures demonstrate distinct execution characteristics during inference due to variations in computational graph topology, synchronization behavior, branching patterns, and computational complexity. These factors influence multiple stages of execution, including preprocessing overhead, model computation time, and post-processing latency.

Recent studies on neural-network reverse engineering and model extraction have shown that architectural hints, hardware traces, and query observations can leak information about deployed neural networks [5], [6], [9], [10]. As a result, different architectures may produce distinct latency distributions during inference. In black-box deployment settings, an adversary may repeatedly query the model and analyze these timing variations to infer structural information about the underlying architecture.

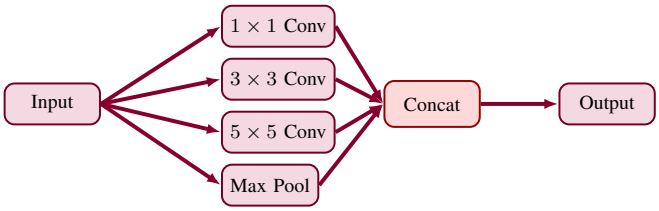

Fig. 3. Simplified Inception-style multi-branch module with concatenation merge.

D. Black-Box Setting

The experiment presumes a black-box deployment environment in which the attacker has no visibility into the internal architecture, parameters, data used in training, or implementation insights of the deployed neural network model. The target is accessible solely through a request-response interface. Despite complete restriction from internal access, the adversary can repeatedly query the model and observe inference response times. These timing records are further analyzed in order to infer structural characteristics of the underlying model.

III. Threat Model

The threat model utilized in this work presumes an adversary engaging with a remotely hosted neural network system through an API interface. The attacker does not possess any information regarding internal mechanics, architecture, model weights, training dataset, hardware specifics, or implementation details of the deployed architecture.

However, the attacker is sufficiently capable of repeatedly requesting the target system and recording inference response times with high accuracy. By observing latency behavior across multiple readings, the attacker attempts to deduce structural properties of the underlying neural network. The main objective of the adversary is to perform architecture fingerprinting using latency-derived features in a black-box deployment environment [4], [7].

IV. Methodology

A. Model Selection

For the purpose of this academic research, four convolutional neural network (CNN) architectures were selected:

- ResNet-18
- ResNet-50
- VGG-16
- InceptionV3

These models were loaded from `torchvision.models` [1], [2], [14] and were selected because they represent significantly different computation graph structures. The first model, VGG, primarily follows a sequential structure, whereas ResNet introduces skip connections and residual blocks that create branch-and-merge computation paths. InceptionV3 further uses parallel branches and concatenation, increasing the structural complexity.

These architectural differences make the selected models suitable for evaluating whether computational graph structure influences inference latency and timing behavior.

Fig. 4. Latency measurement and feature extraction pipeline utilized during inference experiments.

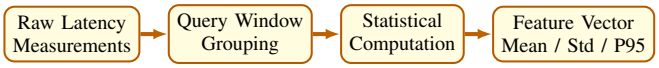

Fig. 5. Statistical feature extraction pipeline used for latency fingerprint generation.

B. Latency Measurement

Inference latency measurements were conducted on an AMD Ryzen 7 5800H CPU with 16GB RAM running Ubuntu 22.04 and PyTorch 2.2. All models received identical input tensors of size `(1,3,224,224)` to ensure consistent evaluation conditions. Each model was switched to evaluation mode using `model.eval()` in order to disable training-specific behavior during inference. In addition, `torch.no_grad()` was utilized to disable gradient computation and reduce unnecessary overhead during latency measurement. A warm-up inference pass was executed prior to timing measurements to reduce initialization overhead.

Each architecture executed 100 inference cycles, with individual inference times recorded using `time.perf_counter()`. The duration of every forward pass was measured in milliseconds, producing a latency distribution for each architecture. Statistical features including mean latency, standard deviation, minimum latency, maximum latency, and P95 latency were subsequently extracted from these distributions. Figure 4 illustrates the latency measurement workflow utilized during the experiments.

C. Feature Extraction

Raw latency measurements were transformed into statistical timing features for architecture fingerprint generation. Instead of relying on individual latency observations, the proposed methodology summarized inference behavior using statistical descriptors including mean latency, standard deviation, minimum latency, maximum latency, and P95 latency. These features were selected because they capture both average execution behavior and latency variability across different neural network architectures.

Latency measurements were grouped across multiple inference queries in order to generate stable feature representations for classification. The resulting statistical feature vectors were subsequently associated with their corresponding architecture labels and used for downstream classification tasks.

Figure 5 illustrates the statistical feature extraction pipeline utilized throughout the experiments.

D. Classification Approach

Latency-derived feature vectors were used to train lightweight architecture classification models. Each feature vector represented the statistical timing characteristics of a specific neural network architecture including ResNet-18, ResNet-50, VGG-16, and InceptionV3.

Fig. 6. Latency-based architecture classification workflow utilized during experiments.

TABLE I
LATENCY STATISTICS FOR DIFFERENT ARCHITECTURES

Model	Mean (ms)	Std Dev (ms)	P95 (ms)
ResNet-18	42.0168	3.9156	51.0841
ResNet-50	128.7198	14.5781	159.9856
VGG-16	297.7229	114.8033	604.6216
InceptionV3	159.8671	75.5069	346.9756

The dataset was divided into training and testing subsets using an 80/20 split. Prior to classification, feature normalization was applied in order to reduce scale variation across different statistical descriptors. K-Nearest Neighbors (KNN) and Logistic Regression classifiers were subsequently trained and evaluated on the extracted latency features. KNN was evaluated using $k = 5$, while Logistic Regression was configured with standard L2 regularization.

Classification accuracy, confusion matrices, and classification reports were utilized to evaluate the effectiveness of latency-based architecture fingerprinting. Figure 6 illustrates the classification workflow implemented during the experiments.

V. RESULTS

A. Latency Measurements

Table I summarizes the observed latency characteristics across all evaluated architectures.

Table I shows that different neural network architectures exhibit distinct inference latency characteristics. ResNet-18 produced the lowest average latency at 42.0168 ms with relatively low variance, whereas VGG-16 exhibited the highest average latency and the largest latency spread. InceptionV3 and ResNet-50 demonstrated intermediate latency behavior with higher variability compared to ResNet-18.

The P95 latency values further highlight differences in execution behavior across architectures. VGG-16 exhibited the highest tail latency at 604.6216 ms, indicating substantial latency variation during repeated inference execution. InceptionV3 also demonstrated relatively large latency fluctuations compared to ResNet-based architectures.

Figure 7 illustrates the separation between latency distributions across the evaluated architectures. These differences are influenced by architectural properties including network depth, parameter count, branching structure, and computational complexity.

B. Classification Accuracy

Table II presents the architecture identification performance achieved using latency-derived features.

Table II summarizes the classification accuracy achieved using latency-derived statistical features. The KNN classifier

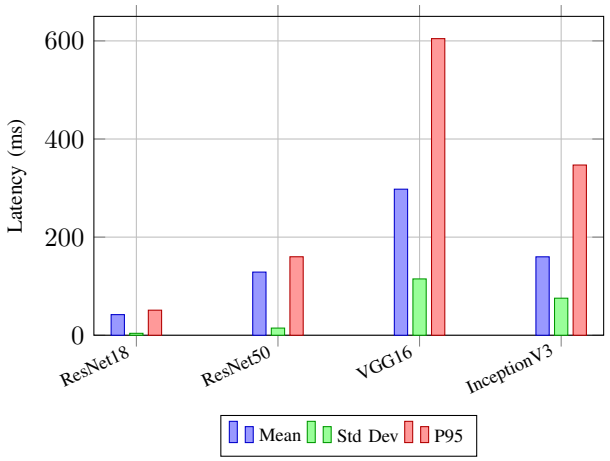

Fig. 7. Latency statistics observed across the evaluated architectures.

TABLE II
MODEL IDENTIFICATION ACCURACY

Classifier	Accuracy (%)	Configuration
KNN	86.25	$k = 5$
Logistic Regression	80.00	L2 regularization

achieved the highest accuracy of 86.25%, outperforming Logistic Regression, which achieved 80.00% accuracy. These results indicate that non-linear neighborhood-based classification methods may better capture variations in latency distributions across architectures.

Table III provides a detailed classification report for both classifiers. ResNet-18 achieved perfect precision and recall across both models, indicating highly distinguishable latency characteristics. VGG-16 also demonstrated strong classification performance due to its significantly different latency distribution. InceptionV3 achieved comparatively lower recall values, suggesting partial overlap between its latency characteristics and those of other architectures.

Overall, the results demonstrate that statistical timing features extracted from inference latency contain sufficient discriminative information for neural network architecture fingerprinting in black-box settings.

C. Effect of Query Count

Looking at Table IV along with Figure 9, it can be observed that increasing the number of queries generally improves classification accuracy. Performance increased from 80.00% using a single query to 89.1% when twenty queries were utilized. Although the improvement is moderate, the overall trend indicates that additional timing observations improve architecture separability.

One possible explanation for the improved stability at higher query counts is that multiple timing samples help reduce the effect of transient execution noise and system-level timing fluctuations. Minor variations observed at intermediate query counts may be influenced by background processes or small execution inconsistencies during inference.

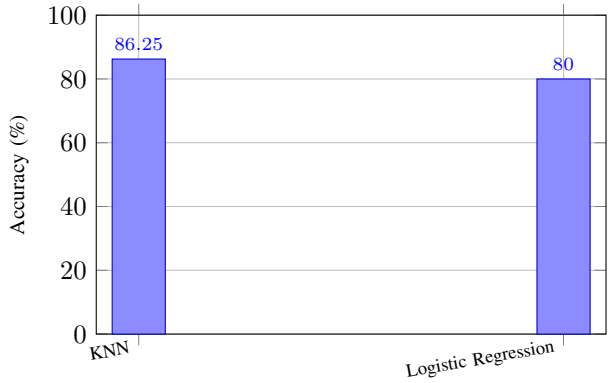

Fig. 8. Architecture identification accuracy using latency-derived statistical features.

TABLE III
CLASSIFICATION REPORT FOR MODEL IDENTIFICATION

Classifier	Class	Precision	Recall	F1-score	Support
KNN	ResNet-18	1.0000	1.0000	1.0000	20
KNN	ResNet-50	0.7917	0.9500	0.8636	20
KNN	VGG-16	0.8261	0.9500	0.8837	20
KNN	InceptionV3	0.8462	0.5500	0.6667	20
LR	ResNet-18	1.0000	1.0000	1.0000	20
LR	ResNet-50	0.6250	1.0000	0.7692	20
LR	VGG-16	0.8333	1.0000	0.9091	20
LR	InceptionV3	1.0000	0.2000	0.3333	20

The results further suggest that repeated timing observations strengthen the ability to distinguish neural network architectures in black-box environments. Overall, the findings indicate that latency patterns, combined with statistical feature extraction, can effectively reveal structural characteristics of deployed neural network systems.

VI. DISCUSSION

The experimental results demonstrate that inference latency contains identifiable architecture-dependent timing patterns. Different neural network architectures exhibited distinct latency distributions due to variations in computational graph topology, execution flow, synchronization behavior, and computational complexity.

The results further indicate that latency variability, in addition to average inference time, contributes significantly to architecture separability. Architectures with deeper or more computationally intensive structures demonstrated broader latency distributions compared to relatively lightweight residual architectures.

Despite the promising results, the experiments were conducted in a controlled CPU-based environment using a limited set of convolutional neural network architectures. Real-world deployment conditions including heterogeneous hardware, background workloads, network jitter, and cloud scheduling behavior may influence latency characteristics and affect fingerprinting performance.

In addition, the current study utilized relatively simple machine learning classifiers and statistical timing descriptors. Future work may investigate more advanced temporal feature

TABLE IV
ACCURACY VS NUMBER OF QUERIES

Queries	Accuracy (%)
1	80.0
3	85.2
5	81.2
10	87.5
20	89.1

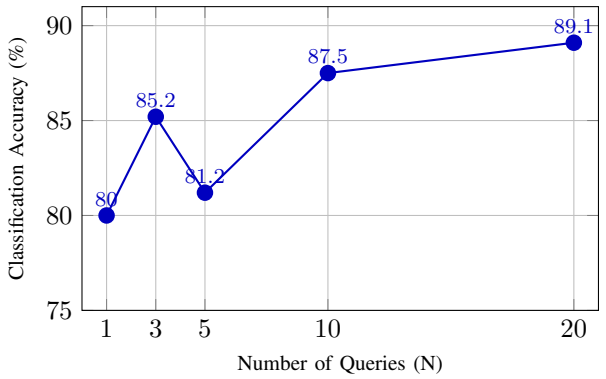

Fig. 9. Query count versus attack success rate for latency-based model fingerprinting.

extraction techniques, transformer-based architectures [17], and defensive mechanisms designed to mitigate timing-based information leakage.

The implementation, latency measurement scripts, and experimental configurations associated with this study will be made publicly available through a GitHub repository in order to support reproducibility and further research.

VII. CONCLUSION

This study demonstrated that inference latency can act as a practical side-channel for neural network architecture fingerprinting in black-box deployment environments. Experimental results showed that different CNN architectures produce distinguishable latency distributions due to variations in computational graph structure and execution behavior.

Statistical timing features including mean latency, standard deviation, and tail latency were successfully utilized for architecture classification using lightweight machine learning models. Among the evaluated classifiers, KNN achieved the highest architecture identification accuracy of 86.25%.

The experiments further demonstrated that increasing the number of timing observations improved classification performance by reducing measurement variability. These findings indicate that inference latency can leak meaningful structural information about deployed neural network models.

Overall, the results highlight the security implications of timing leakage in machine learning systems and demonstrate the feasibility of latency-based neural network fingerprinting in black-box deployment settings.

VIII. FUTURE WORK

Future work may extend the current study by evaluating latency-based architecture fingerprinting across heterogeneous hardware environments including GPUs, edge accelerators, cloud inference servers, and mobile devices. Since hardware-specific optimizations and scheduling behavior can significantly influence inference timing, analyzing cross-platform consistency remains an important direction for improving the generalizability of the proposed methodology.

Further investigation may also incorporate larger and more diverse neural network families including transformer architectures, hybrid models, and quantized networks. Expanding the feature space beyond basic statistical descriptors to include temporal sequences, frequency-domain features, and higher-order timing correlations may improve architecture separability and attack robustness.

Another potential research direction involves studying the impact of real-world deployment conditions such as network jitter, concurrent workloads, caching effects, and asynchronous execution pipelines on timing leakage. Evaluating latency-based fingerprinting in realistic cloud and API-driven inference environments would provide deeper insight into the practical feasibility of side-channel attacks against deployed machine learning systems.

Finally, future research may focus on the development of defensive mechanisms designed to mitigate timing leakage during inference. Techniques such as randomized execution delays, timing obfuscation, workload equalization, and constant-time inference strategies could be explored to reduce architecture-dependent side-channel information while maintaining acceptable inference performance.

REFERENCES

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[2] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] P. Kocher, "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems," in *Proceedings of the Annual International Cryptology Conference (CRYPTO)*, 1996.

[4] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "PRADA: Protecting Against DNN Model Stealing Attacks," in *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.

[5] L. Batina, S. Bhasin, D. Jap, and S. Picek, "CSI NN: Reverse Engineering of Neural Network Architectures Through Electromagnetic Side Channel," in *Proceedings of the USENIX Security Symposium*, 2019.

[6] X. Hu, L. Liang, S. Li, L. Deng, P. Zuo, Y. Ji, X. Xie, Y. Ding, C. Liu, T. Sherwood, and Y. Xie, "DeepSniffer: A DNN Model Extraction Framework Based on Learning Architectural Hints," in *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.

[7] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff Nets: Stealing Functionality of Black-Box Models," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[8] N. Carlini et al., "The Secret Sharer: Measuring Unintended Neural Network Memorization and Extractability," in *Proceedings of the USENIX Security Symposium*, 2019.

[9] J. Wei et al., "Machine Learning Model Extraction Through Hardware Side Channels," in *Proceedings of the IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2020.

[10] M. Hua, Y. Wang, S. Wang, and H. Qian, "Reverse Engineering Convolutional Neural Networks Through Side-Channel Information Leaks," in *Proceedings of the Design Automation Conference (DAC)*, 2018.

[11] Y. Kang et al., "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.

[12] C. Zhang et al., "Understanding Deep Learning System Performance," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2017.

[13] S. Han et al., "EIE: Efficient Inference Engine on Compressed Deep Neural Network," in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2016.

[14] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *International Conference on Learning Representations (ICLR)*, 2015.

[15] C. Szegedy et al., "Going Deeper with Convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[16] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," in *Proceedings of the International Conference on Pattern Recognition (ICPR)*, 2016.

[17] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.